

Понятие ОС. Назначение ОС. Требования к аппаратуре.

Операционная система — это организованная совокупность процессов, которая:

- действует как интерфейс между аппаратурой компьютера и пользователями;
- предоставляет средства для:
 - проектирования,
 - программирования,
 - отладки,
 - сопровождения программ;
- управляет распределением ресурсов для эффективной работы системы.

Назначение ОС: три основные цели ОС:

1. Обеспечение максимальной эффективности системы

- загрузка устройств (распараллеливание CPU и ввода/вывода),
- высокая пропускная способность,
- минимальное время ответа.

2) Обеспечение максимальных удобств пользователю

- системы программирования,
- ввод/вывод → файлы → базы данных,
- сервисные программы,
- работа с библиотеками,
- средства секционирования и объединения программ,
- диагностика.

3) Обеспечение максимальной надежности

- организация защиты,
- обработка сбоев,
- средства восстановления при отказах,
- контрольные точки.

Требования к аппаратуре

Для нормальной работы ОС аппаратура должна поддерживать:

- способы адресации памяти;
- аппаратную организацию защиты (интервалы адресов, ключи защиты);
- наличие привилегированных команд и режимов супервизор / задача (пользовательский режим);
- наличие аппаратных прерываний.

2. Классификация ОС

1. по особенностям алгоритмов управления ресурсами
2. по особенностям аппаратных платформ
3. по особенностям областей использования

2.1 По особенностям алгоритмов управления ресурсами

Поддержка многозадачности

По числу одновременно выполняемых задач:

- **Однозадачные ОС:** Выполняют только одну задачу в каждый момент времени. Примеры: MS-DOS, MSX.

Многозадачные ОС:

Могут выполнять несколько задач одновременно.

Примеры: ОС ЕС, OS/2, UNIX, Windows 95.

2.2 По особенностям аппаратных платформ

Различают ОС для:

- персональных компьютеров,

- мини-компьютеров,
- мейнфреймов,
- кластеров,
- сетей ЭВМ.

2.3 По особенностям областей использования

Для каких целей предназначена ОС.

Системы пакетной обработки. Пример: ОС ЕС

Назначение:

выполнение задач, не требующих быстрого ответа.

Системы разделения времени

Примеры: UNIX, VMS. Назначение:

1. интерактивная работа пользователя с программой,
2. малое время отклика.

Системы реального времени

Примеры: QNX, RT/11. Назначение:

1. управление техническими объектами,
2. строгое соблюдение временных ограничений.

2.4 По особенностям методов построения

Монолитное ядро — все основные функции ОС находятся в одном ядре и работают в привилегированном режиме.

Микроядро — в ядре находятся только базовые функции, остальные службы работают как отдельные процессы.

Объектно-ориентированный подход — компоненты ОС представлены как объекты с определёнными интерфейсами.

Распределённая организация — части ОС работают на разных компьютерах и взаимодействуют по сети.

3. Процессы

3.1 Понятие процесса

- Процесс — это программа в процессе выполнения.
- Процесс включает код программы, данные и состояние выполнения.
- Процесс является «открытой системой», так как на него могут влиять другие процессы и ОС.
- Выполнение процесса может быть прервано.

3.2. Реализация процессов

- Каждый процесс описывается специальной структурой — **дескриптором процесса**.
- Дескриптор содержит:
 - состояние процесса,
 - сохранённые значения регистров,
 - информацию о ресурсах процесса.
- По дескриптору ОС управляет выполнением процесса.

3.3. Общение между процессами

Процессы могут:

- совместно использовать ресурсы,
- обмениваться данными.
- Для этого используются специальные механизмы межпроцессного взаимодействия.

- При совместной работе возможны ошибки из-за одновременного доступа к данным, поэтому требуется синхронизация.

3.4. Синхронизация процессов

- Синхронизация нужна, чтобы несколько процессов не использовали один ресурс одновременно.
- В каждый момент времени критический ресурс может использовать только один процесс.

3.4.1. Блокировка памяти

- Процесс запрещает другим процессам доступ к общей области памяти.
- Используются специальные переменные-флаги.
- Простой метод, но может приводить к ошибкам и зависаниям.

3.4.2. Операция "проверка и установка"

- ☐ Это атомарная аппаратная операция.
- ☐ Проверяет значение переменной и сразу устанавливает новое.
- ☐ Используется для организации взаимного исключения.
- ☐ Не допускает одновременного входа в критическую секцию.

3.4.3. Семафоры

- ☐ Семафор — это целая переменная (S), изменяемая только операциями **P** и **V**.
- ☐ **P(S)** уменьшает значение и может блокировать процесс.
- ☐ **V(S)** увеличивает значение и может разблокировать процесс.
- ☐ Используются для управления доступом к ресурсам и синхронизации.

3.4.4. Почтовые ящики

- ☐ Используются для обмена сообщениями между процессами.
- ☐ Процесс может отправить сообщение в ящик и другой процесс может его получить.
- ☐ Процессы не обращаются напрямую к памяти друг друга.

3.4.5. Порты

- Порт — это точка связи для передачи сообщений.
- Процессы отправляют и получают данные через порты.
- Часто используются в сетевых и распределённых системах.

3.4.6. Монитор Хоара

- Монитор — это высокоуровневый механизм синхронизации.
- Объединяет данные и процедуры доступа к ним.
- Гарантирует, что в монитор в один момент входит только один процесс.
- Упрощает программирование и уменьшает ошибки синхронизации.

4. Понятие нити

это набор команд, который может получать время процессора OR ☐ **Нить (поток)** — это часть процесса, которая выполняется процессором.

- У нити есть свои регистры, стек и состояние.
 - Нити одного процесса разделяют память и ресурсы.
- У нити собственные: Программный счётчик, стек, регистры, нити-потомки, состояние
- Нити разделяют: адресное пространство, глобальные переменные, открытые файлы, таймеры, семафоры

4.1. Реализация потоков в Windows

- Потоки создаются с помощью функции **CreateThread** (Win32 API).
- Указывается функция, которая будет выполняться потоком.
- Поток может запускаться сразу или быть приостановленным.
- Для управления используются функции **ResumeThread** и **SuspendThread**.
- Поток создаётся с помощью функции **API Function**

function CreateThread(

Attr: Pointer; //адрес атрибутов безопасности
Stack:Dword; //размер стека для потока
Start:Pointer; //начальный адрес потока
par:Pointer; //аргументы потока
flag:Dword; // флаг создания
var ID:Dword //идентификатор потока

);THandle; //дескриптор потока

Существуют две модели применения потоков:

асимметричная — потоки решают разные задачи и, как правило, не разделяют ресурсы;

симметричная — потоки выполняют одну и ту же задачу, разделяют ресурсы и используют один и тот же код.

4.2. Средства синхронизации потоков

- Синхронизация нужна для защиты общих данных.
- Она не допускает одновременный доступ нескольких потоков к ресурсу.

4.2.1. Метод Synchronize

- Используется в классе **TThread**.
- Позволяет выполнять код потока в главном потоке программы.
- Применяется для безопасной работы с интерфейсом.

4.2.2. Критическая секция

Критическая секция — это участок кода с общим ресурсом.

- В неё может войти только один поток.
 - Используется для защиты памяти и переменных.
- Ог Это участок кода, в котором происходит обращение к критическому ресурсу.
- В Win32 API синхронизация критических секций может осуществляться с помощью глобальной записи специального вида типа **TRTLCriticalSection**.
- Инициализация выполняется процедурой

```
InitializeCriticalSection(var lpCriticalSection:  
TRTLCriticalSection);
```

- Перед входом в критическую секцию:

```
EnterCriticalSection(var lpCriticalSection:  
TRTLCriticalSection);
```

- При выходе:

```
LeaveCriticalSection(var lpCriticalSection:  
TRTLCriticalSection);
```

По окончании работы:

```
DeleteCriticalSection(var lpCriticalSection:  
TRTLCriticalSection);
```

4.2.3. Семафоры

- Семафор ограничивает количество потоков, имеющих доступ к ресурсу.

- Использует счётчик разрешений.

- Применяется для синхронизации и управления очередями.

Ог Функции для работы с семафорами и мьютексами.

Описание Semaph_Mutex

Пример Sem_Mutex

4.2.4. Мьютексы

- Мьютекс разрешает доступ к ресурсу только одному потоку.

- Это двоичный семафор.

- Используется для строгого взаимного исключения.

5. Тупики

5.1 Понятие тупика

Определение: Тупик (deadlock) — ситуация, в которой два или более процессов находятся в состоянии бесконечного ожидания ресурсов, удерживаемых друг другом.

Неформальный пример: Два процесса ждут друг друга, как два автомобиля на перекрёстке, никто не может проехать первым.

Формальные условия Коффмана–Элфика–Шошани (четыре необходимых условия наличия тупика):

1. Взаимоисключение (Mutual Exclusion): процессы требуют предоставления им права монопольного управления ресурсами, которые им выделяются;

2. Удержание и ожидание (Hold and Wait): процессы удерживают за собой уже выделенные ресурсы, требуя выделения дополнительных ресурсов;

3. Неперераспределяемость (No Preemption): ресурсы нельзя отобрать у процесса, пока он не завершит работу с ними;

4. Круговое ожидание (Circular Wait): существует круговая цепь процессов, в которой каждый удерживает один или более ресурсов, требующихся другому.

Четыре подхода к решению проблемы тупиков:

1. Предотвращение (Prevention): При предотвращении тупиков целью явл. обеспечение условий, исключающих возможность возникновения тупиковых ситуаций.

Достоинство: полное исключение тупиков.

Недостаток: такой подход часто приводит нерациональному использованию ресурсов.

2. Обход (Avoidance): Цель средств обхода тупиков заключается в том, чтобы можно было предусмотреть менее жесткие ограничения, чем в случае предотвращения тупиков.

Методы обхода тупиков учитывают возможность возникновения тупика, однако в этом случае принимаются меры по аккуратному обходу тупика.

3. Обнаружение (Detection): Методы обнаружения тупиков применяются в системах, допускающих возможность возникновения тупиков.

Когда это происходит, система обнаруживает тупик программным путём и принимает меры для вывода из тупика (например перераспределяя ресурсы).

Выход из тупика может выполняться автоматически или под управлением оператора.

4. Восстановление (Recovery): Методы восстановления после тупиков применяются для устранения тупиковых ситуаций. Например:

- восстановление при помощи принудительной выгрузки;
- восстановление через откат;
- восстановление путем уничтожения одного или нескольких процессов.

5.1.1. Алгоритмы предотвращения тупиков

Выделение всех необходимых ресурсов заранее

Выделение ресурсов по порядку номеров

Метод Габермана (Haberma's Algorithm)

Алгоритм банкира (Banker's Algorithm)

5.1.2. Закрепление ресурсов

Суть: Процесс запрашивает все необходимые ресурсы сразу до начала выполнения.

Плюсы:

- Простота реализации
- Гарантированное отсутствие тупиков

Минусы:

- Низкая эффективность использования ресурсов
- Возможность простоев процессов

5.1.3. Присваивание фиксированных номеров

Идея: Каждому ресурсу присваивается уникальный номер. Процесс может запрашивать ресурсы только в порядке возрастания номеров.

Пример: Ресурсы: R1 (№1), R2 (№2), R3 (№3)

Процесс может запросить R1 → R3, но не R3 → R1.

Преимущества:

- Исключает круговое ожидание
- Простота реализации

Недостатки:

- Жёсткое ограничение на порядок запросов
- Не всегда соответствует логике работы приложения

5.1.4. Метод Габермана

Основа: Использование ориентированного графа процессов и ресурсов.

Правило: Ресурс выделяется, если в графе не образуется цикл после его выделения.

Алгоритм:

1. Строится граф, где:
 - а. Узлы — процессы и ресурсы
 - б. Дуга от процесса к ресурсу: запрос
 - с. Дуга от ресурса к процессу: выделение
2. При запросе ресурса проверяется, не образуется ли цикл.
3. Если цикл есть — запрос отклоняется (процесс блокируется).

Пример из презентации:

P1 → A, P2 → A (блокировка), P3 → B, P4 → D (цикл → блокировка)

Правило Габермана гласит:

Состояние явля. опасным, если граф содержит цикл.

Алгоритм вызывается при каждом запросе и возврате ресурса.

6. Распределение времени процессора

6.1. Цели планирования

Справедливость — каждому процессу должна быть предоставлена честная доля процессорного времени.

Эффективность — минимизация простоев CPU, максимальная пропускная способность.

Отзывчивость — быстрое время отклика в интерактивных системах.

Предсказуемость — гарантированное время выполнения для систем реального времени.

Баланс нагрузки — равномерное использование всех системных ресурсов (CPU, I/O, память).

Учёт приоритетов — возможность выделять больше времени важным или фоновым задачам.

Классификация алгоритмов планирования:

Алгоритмы планирования можно разделить на две категории согласно их поведению после прерываний.

- **с переключениями**, называемого также **приоритетным** планированием, выбирают процесс и позволяют ему работать некоторое максимально возможное фиксированное время. Категории алгоритмов планирования
- **без переключений**, иногда называемого также **неприоритетным** планированием, выбирают процесс и позволяют ему работать вплоть до блокировки

Особенности планирования:

В системах пакетной обработки приемлемы алгоритмы без переключений или с переключениями, но с большим временем, отводимым каждому процессу. Такой метод уменьшает количество переключений между процессами и улучшает эффективность.

- **В интерактивных системах** необходимы алгоритмы планирования с переключениями, чтобы предотвратить захват процессора одним процессом.
- **В системах с ограничениями реального времени** приоритетность, не всегда обязательна, поскольку процессы знают, что их время ограничено, и быстро выполняют работу, а затем блокируются.

Характеристики планирования:

Пропускной способностью называется число выполненных системой задач в час. В любом случае 50 задач в час лучше, чем 40 задач в час.

Оборотное время — статистически усредненное время от момента получения задачи до ее выполнения.

время отклика - время между введением команды и получением результата.

Алгоритм планирования, максимизирующий пропускную способность, не обязательно минимизирует обратное время.

6.2. Планирование в системах пакетной обработки

Характеристики среды:

- Нет интерактивных пользователей.
- Задачи выполняются “пакетами”.
- Основные метрики: пропускная способность, обратное время.

Алгоритмы:

- **FCFS (First-Come, First-Served) «Первым пришел — первым обслужен».**
 - Простейший, невытесняющий.
 - Проблема: “эффект конвоя” (долгие задачи задерживают короткие).
- **SJF (Shortest Job First) Кратчайшая задача - первая.**
 - Выбирается задача с наименьшим временем выполнения.
 - Минимизирует среднее время ожидания.
 - Невытесняющий вариант: Non-Preemptive SJF.
- **SRT (Shortest Remaining Time) “Наименьшее оставшееся время выполнения.”**
 - Вытесняющая версия SJF.
 - Прерывает текущую задачу, если появилась более короткая.

6.3.1. Планирование в интерактивных системах

Циклическое планирование.

- Каждому процессу выделяется **квант** времени.
- Если задача не завершена — перемещается в конец очереди.
- Ключевой параметр: размер кванта (слишком большой → похоже на FCFS, слишком маленький → большие накладные расходы).

Приоритетное планирование.

- Каждому процессу присваивается приоритет.
- Готовый процесс с наивысшим приоритетом выполняется первым.

- Проблема: голодание низкоприоритетных задач → решение: старение (увеличение приоритета со временем).
- **«Самый короткий процесс - следующий».**
- **Гарантированное планирование.**
 - Каждый процесс получает 1/n процесс-ного времени.
- **Лотерейное планирование.**
 - Процессам выдаются “лотерейные билеты”.
 - Планировщик случайно выбирает билет → владелец выполняется.
- **Справедливое планирование.**

6.4. Планирование в системах реального времени

Системы реального времени делятся на **жесткие системы реального времени, гибкие системы реального времени**. Внешние события, на которые система должна реагировать, можно разделить на

- **периодические;**
- **непериодические.**

Если все потоки могут быть своевременно обработаны, системы реального времени, называются **поддающимися планированию** или **планируемыми**.

Алгоритмы планирования для систем реального времени могут быть **статическими** - все решения планирования принимаются заранее, еще до запуска системы, **динамическими** - решения планирования принимаются по ходу дела.

7. Управление памятью

7.1. Методы распределения памяти без использования диска

7.1.1. Фиксированными разделами

Суть: Память делится на несколько разделов фиксированного размера.

Схема работы:

- Каждому процессу выделяется целый раздел.
- Разделы могут быть одинаковыми или разными.

Варианты очередей:

Общая очередь — один список ожидания для всех разделов.

Отдельные очереди — свой список для каждого раздела.

Преимущества: простота реализации, низкие накладные расходы.

Недостатки: внутренняя фрагментация, негибкость.

Пример: ОС ранних мейнфреймов (IBM OS/360).

7.1.2. Динамическими разделами

Суть: Разделы создаются динамически под размер процесса.

Схема работы:

- При загрузке процесса ищется свободный блок достаточного размера.
- После завершения процесса блок освобождается.

Проблема: Внешняя фрагментация- множество мелких свободных блоков, которые невозможно использовать.

Алгоритмы выбора блока:

First Fit — первый подходящий.

Best Fit — наименьший подходящий.

Worst Fit — наибольший подходящий.

Пример: ОС для мейнфреймов с пакетной обработкой.

7.1.3. Перемещаемыми разделами

Суть: Возможность перемещать занятые блоки для устранения фрагментации.

Процедура сжатия (компактизация):

Все занятые блоки сдвигаются в начало памяти.

Свободное пространство объединяется в один блок.

Преимущества: устранение внешней фрагментации.

Недостатки: высокие накладные расходы, сложность реализации.

Пример: ОС реального времени, встроенные системы.

7.2. Методы распределения памяти с использованием диска

7.2.1. Свопинг

Суть: Процесс полностью выгружается на диск, освобождая оперативную память.

Применение: Когда RAM недостаточно для всех активных процессов.

Проблемы: Большие задержки из-за медленного диска.

7.2.2. Сегментация

Суть: Виртуальное адресное пространство делится на сегменты (логические единицы: код, данные, стек).

Адрес: [номер сегмента, смещение].

Таблица сегментов: хранит базовый адрес и длину каждого сегмента.

Преимущества: логическая структура, защита на уровне сегментов.

Недостатки: внешняя фрагментация.

7.2.3. Страничная организация

Суть: Виртуальная память и физическая память делятся на страницы/фреймы фиксированного размера.

Адрес: [номер страницы, смещение].

Таблица страниц: отображает виртуальные страницы на физические фреймы.

Преимущества: нет внешней фрагментации, простота управления.

Недостатки: внутренняя фрагментация, накладные расходы на таблицы.

7.2.3.1. Сегментно-страничная организация

Суть: Комбинация сегментации и страничной организации.

Адрес: [номер сегмента, номер страницы, смещение].

Две таблицы: таблица сегментов → таблицы страниц.

Преимущества: логическая структура + эффективное управление памятью.

Недостатки: сложность, двойное преобразование адреса.

7.3. Понятие виртуальной памяти

Определение: Механизм, предоставляющий процессу иллюзию большой непрерывной памяти. OR программно-аппаратный механизм, который позволяет выполнять программы, размер которых превышает объём оперативной памяти. **Цели:**

- Изоляция процессов.
- Возможность выполнения программ, превышающих размер RAM.
- Эффективное использование памяти.

Основные концепции: страничная организация, подкачка, кэширование.

7.4. Иерархия запоминающих устройств. Принцип кэширования данных

Иерархия: Регистры → Кэш L1/L2/L3 → RAM → Диск (SSD/HDD) → Сеть.

Принцип локальности:

- Временная: недавно использованные данные будут использоваться снова.
- Пространственная: данные рядом с используемыми будут использоваться вскоре.

Кэширование: Хранение часто используемых данных на более быстрых уровнях памяти.

7.5. Фрагментация

Фрагментация — появление неиспользуемых участков памяти, которые нельзя задействовать под новые данные.

Тип	Описание	Где возникает	Методы борьбы
Внутренняя	Незаполненное пространство внутри блока	При страничной организации	Не устраняется (размер страницы фиксирован аппаратно)
Внешняя	Мелкие куски между блоками	При сегментации	Уплотнение (компактизация) — перемещение сегментов и обновление таблиц

Размер страницы влияет на фрагментацию:

- Теоретически оптимальный размер ≈ 128 байт,
- На практике — 4 КБ (баланс между фрагментацией и скоростью подкачки).

7.6. Распределение ресурсов при сегментации и страничной организации

7.6.1. Распределение памяти

- Сегментация: Выделяется целый сегмент под задачу.
- Страничная организация: Выделяются отдельные страницы.

- **Таблицы:** TLB (Translation Lookaside Buffer) для ускорения преобразования адресов.
- OR... При страничной — учёт свободных кадров; при сегментации — списки свободных областей, алгоритмы поиска (first fit / best fit / buddy-system).

7.6.2. Стратегия подкачек

- Подкачка по запросу (Demand Paging): Страница загружается только при обращении.
- Предварительная подкачка (Prepaging): Загрузка страниц заранее на основе прогноза.
- Ожидание (Swapping): Выгрузка целого процесса.
- OR... Подкачка по запросу (on-demand) и опережающая подкачка (pre-paging / prefetch), ориентированная на прогноз локальностей

7.6.3. Правила вытеснения страниц

Алгоритмы замещения: FIFO, случайный, LRU и его приближения, LFU/MFU и др.; цель — минимизировать число промахов (page faults).
OR.. FIFO (First-In, First-Out)
LRU (Least Recently Used)
Clock (Second Chance)
OPT (Optimal) — теоретический идеал.
Проблема «бесконечного свопинга» (Thrashing).

8. Управление распределёнными ресурсами

8.1 Базовые примитивы передачи сообщений в распределённых системах.

Передача сообщений = основа взаимодействия в распределённых системах: нет общей памяти, процессы общаются через send/receive по сети.

Базовые характеристики примитивов:

- адресация (кому отправляем);
- блокирующие / неблокирующие;
- буферизация;
- надёжность доставки.

8.1.1 Способы адресации

Прямая адресация: Отправитель явно указывает получателя: send(P2, msg), receive(P1, msg).

Косвенная адресация: Передача через почтовый ящик / очередь / порт: send(mailboxA, msg), receive(mailboxA, msg); процессы могут меняться, имя — у ящика.

Групповая / multicast-адресация:

Сообщение отправляется «группе», доставляется всем её участникам, отправитель может не знать конкретных получателей

8.1.2 Блокирующие и неблокирующие примитивы

Блокирующий send

Отправитель ждёт, пока сообщение будет передано (или хотя бы скопировано в ядро / очередь).

Блокирующий receive

Процесс блокируется, пока сообщение не придёт.

Неблокирующий send

Возврат управления сразу после постановки сообщения в очередь / начало копирования; передача идёт «в фоне».

Неблокирующий receive

Процесс сразу возвращается, даже если сообщений нет (обычно с признаком «ничего не пришло»).

8.1.3 Буферизуемые и небуферизуемые примитивы

Буферизуемые (buffered): Между отправителем и получателем есть очередь (буфер); отправитель может продолжать работу, пока буфер не заполнен.

Небуферизуемые (unbuffered, zero-capacity)

Сообщение передаётся только при одновременной готовности send и receive; по сути, синхронный «rendezvous» без промежуточного хранения.

Размер буфера (0, ограниченный, неограниченный) влияет на необходимость блокировки и возможные взаимные блокировки.

8.1.4 Надёжные и ненадёжные примитивы

Надёжные: Гарантируют доставку без потерь и дубликатов (обычно: подтверждения, номера сообщений, тайм-ауты и повторная отправка).

Ненадёжные: Сообщение может потеряться, дублироваться или прийти повреждённым; поверх такого канала прикладной уровень должен сам реализовать контроль целостности и повтор.

9. Вызов удалённых процедур RPC

RPC (Remote Procedure Call) — механизм, позволяющий вызывать процедуры на удалённом сервере так, будто они локальные.

9.1. Концепция удалённого вызова процедур

Клиент вызывает функцию, клиентский стаб (stub) сериализует параметры (маршаллинг), отправляет по сети. Серверный стаб десериализует, выполняет процедуру, возвращает результат. **Прозрачность:** разработчик не видит сеть.

9.2. Базовые операции RPC

Клиент: вызов(параметры) → стаб → сеть.

Сервер: приём → стаб → выполнить(параметры) → результат → сеть.

Обработка ошибок: таймауты, повторы, осиротевшие вызовы.

9.3. Этапы выполнения RPC

Клиент: параметры в стек → маршаллинг → отправка.

Сеть: передача пакета (заголовок + данные + контрольная сумма).

Сервер: приём → демаршаллинг → вызов процедуры.

Обратный путь: результат → клиент.

10. Синхронизация в распределённых системах.

10.1. Алгоритм синхронизации логических часов

Логические часы Лампорта решают "что произошло раньше?":

- Каждый процесс имеет счётчик C_i .
- Событие: $C_i += 1$
- Сообщение: отправить C_i , получатель: $C_j = \max(C_j, C_i) + 1$.
- $a < b$, если $C(a) < C(b)$.

10.2 Алгоритмы взаимного исключения

10.2.1. Централизованный алгоритм

Один координатор выдаёт разрешения:

Запрос → координатор → "ОК" (3 сообщения).

Плюсы: простота. Минусы: единая точка отказа.

10.2.2. Распределённый алгоритм

Каждый процесс запрашивает всех ($> n/2$ согласий):

Отказоустойчив (сбой $< n/2$).

Минусы: $\sim n$ сообщений, риск deadlock.

10.2.3. Алгоритм Token Ring

Токен циркулирует по кольцу; держатель входит в КС:

- 0-1 сообщение, справедливость.
- Минусы: задержка $O(n)$, разрыв кольца при сбое.

10.2.4 Неделимые транзакции

2PC (two-phase commit):

Подготовка: "готово?" → все "да".

Коммит: "commit" всем или "abort".

ACID-свойства в распределённых системах.

11. Файловая система

В широком смысле понятие "файловая система" включает:

- совокупность всех файлов на диске;
- наборы структур данных, используемых для управления файлами, такие, например, как каталоги файлов, дескрипторы файлов, таблицы распределения свободного и занятого пространства на диске;
- комплекс системных программных средств, реализующих управление файлами, в частности: создание, уничтожение, чтение, запись, именование, поиск и другие операции над файлами.

11.1. Имена файлов

Иерархическая структура: абсолютный/относительный путь (/dir/subdir/file.ext). Монолитная модель (Unix) или ссылочная. OR.

Файлы идентифицируются именами.

Пользователи дают файлам символьные имена, при этом учитываются ограничения ОС как на используемые символы, так и на длину имени.

Ограничения на длину имени:

- файловой системе FAT – схема 8.3;
- ОС UNIX System V – 14 символов;

- ОС Windows 95 – файловая система VFAT – 256 символов.

Разные файлы могут иметь одинаковые символьные имена:

В этом случае файл однозначно идентифицируется так называемым составным именем, представляющим собой последовательность символьных имен каталогов.

В некоторых системах одному и тому же файлу не может быть дано несколько разных имен, а в других такое ограничение отсутствует.

4. 2. Типы файлов

- обычные файлы
 - текстовые ,
 - двоичные;
- специальные файлы;
- файлы-каталоги.

11.3. Логическая организация файла

Программист имеет дело с логической организацией файла, представляя файл в виде определенным образом организованных логических записей.

Логическая запись - это наименьший элемент данных, которым может оперировать программист при обмене с внешним устройством.

Даже если физический обмен с устройством осуществляется большими единицами, операционная система обеспечивает программисту доступ к отдельной логической записи.

11.4. Физическая организация файла

описывает правила расположения файла на устройстве внешней памяти, в частности на диске. Файл состоит из физических записей - блоков.

Блок - наименьшая единица данных, которой внешнее устройство обменивается с оперативной памятью.

выделяет способ размещения файлов на диске и учет соответствия блоков диска файлам.

Основными критериями эффективности физической организации файлов являются:

- скорость доступа к данным;
- объем адресной информации файла;
- степень фрагментированности дискового пространства;
- максимально возможный размер файла.

ОС — совокупность взаимодействующих процессов (параллелизм → эффективность; отвечает на непредсказуемые запросы).

Процессы vs Потоки (повтор + дополнения)

Процесс: Открытая система (взаимодействие), живёт один раз (новый запуск = новый процесс).

Поток (Thread): Облегчённый процесс в .NET. Имеет процедуру + стек, но делит ресурсы (буфер, данные) с другими потоками одного AppDomain.

Thread t = new Thread(new ThreadStart(MyMethod));

// Без params

t.Start();

// С параметрами

Thread t2 = new Thread(new ParameterizedThreadStart(MyMethodWithObj));

t2.Start(myObject);